
Qonaqzhai

A marketplace for booking Kazakh weddings, corporate events, and traditional ceremonies — built with Go, Flutter, and an AI planner that actually plans.

Bahaidahar · Astana IT University · May 2026

WHERE THIS STARTED

I tried to plan a wedding.

It took three months, six WhatsApp chats, and one deposit I almost didn't get back.
That's why this exists.

//

In Kazakhstan, planning an event still means scrolling Instagram, asking your aunt, and trusting a stranger with a card number.

— Roughly every couple I asked.

What I actually built

A platform that lets a customer **describe an event in plain Russian or Kazakh** and walk away with a booked, paid plan.

Three clients sharing one Go backend. A Claude-powered planner that calls real APIs instead of hallucinating prices.

- **Web** — Next.js 16 for desktop browsing
- **Mobile** — Flutter app, KZ / RU / EN
- **Backend** — five Go microservices behind one gateway
- **AI** — Claude Sonnet 4.6 with tool use, not just chat

Backend, end to end

2,033 LOC of tests

Eight test files across the four services I actually own. Table-driven, `-race` enabled, run on every push. The number isn't impressive on its own — it's that I wrote them at the same time as the code, not after.

The architecture, in one breath

```
client (mobile + web)
  | HTTP
  ▼
gateway :8080 — verifies JWT once, forwards everything else
  |
  ├──> auth      users, JWT, password reset
  ├──> core      vendors, bookings, reviews, photos, FCM
  ├──> payment   cards, PayBox saga
  └──> realtime  WS hub, threads, messages

each owns its own Postgres DB
no shared schema, no cross-DB joins
```

When I split this out from a monolith in phase 9, I didn't believe it would feel cleaner. It does. Each service has one job and one schema.

Why I split it

I could have shipped this as a Go monolith. Honestly, for diploma traffic, I should have.

I split it anyway because I wanted to learn the **real** operational pain of microservices — gRPC contracts that drift, distributed transactions, cold-start chains.

Things I learned the hard way:

- A workspace `replace` directive can break CI silently
- "One DB per service" sounds clean until you need a user's name in three places
- gRPC is great until you try to debug it from a browser

The payment saga

I almost wrote distributed transactions. I'm glad I didn't.

```
core.booking.Pay
  |
  ▼
payment.Charge    (gRPC, sync)
  |
  ▼ on success
core.MarkBookingPaid    (gRPC, sync)
```

Synchronous. Two-step. No event bus, no outbox table.

If the second call fails, the payment row is still captured and a follow-up job reconciles.

It's not the textbook answer. It is the answer that ships.

Mobile, where most users live

```
mobile/lib/features/  
  auth/          vendor_catalog/  booking/  
  ai_chat/       payment/         cards/  
  reviews/      messaging/       notifications/  
  vendor_self/   admin/             onboarding/  
  settings/
```

Thirteen feature folders. Each one is `data → domain → presentation`, Riverpod for state, Dio with a 401-refresh interceptor for HTTP, `go_router` for deep links.

I picked Flutter over React Native because I wanted **one Dart codebase shipping native binaries** — not a JS bridge. Six months in, no regrets, but I still miss hot-reload between Dart and the Go server.

The AI part — the slide I'm asked about most

It's not a chatbot wrapping Claude. It's Claude with **four tools** it can actually call:

```
search_vendors(category, city, budget, date)
check_availability(vendor_id, date)
draft_booking(vendor_id, service_id, guests, date)
estimate_total(items[])
```

When a user says "corporate event, 50 people, Astana, August 20, 1.5M ₸" — Claude calls `search_vendors`, picks the top three by rating, calls `check_availability` for each, drafts a plan, and streams it back. No vendor list invented from thin air.

Stack — opinions, not features

LAYER	WHAT I SHIPPED	WHY, IN ONE SENTENCE
Backend	Go 1.24 + gRPC	Fast, boring, statically linked — runs anywhere
Web	Next.js 16 + React 19	Cache Components finally make App Router feel finished
Mobile	Flutter 3.24	One codebase, native binaries, real animations
State	Riverpod	Compile-safe DI without <code>BuildContext</code> headaches
DB	Postgres per service	Forces real service boundaries, no shared-schema laziness
AI	Claude API + MCP	Tool use is the part that turns chat into a product

Every row is a choice I'd have to defend, so I'm defending them.

Where it sits in the market

PLATFORM	WHERE	WHAT IT DOES	WHAT IT MISSES
The Knot · WeddingWire	US	Big directories, no booking	Kazakh, KZT, traditions
GigSalad · The Bash	US	Vendor commission	No AI, no escrow
Peerspace	US	Venue - only, escrow	Not events, not KZ
Wedding.ru · FlyBride	RU	Paid listings	No transaction flow
Eventie.kz · weddingsalon.kz	KZ	Directories	Web - only, no AI, no payment

So I'm not building the tenth event marketplace. I'm building the first one that speaks Kazakh, takes KZT, and actually books the event.

The hard stuff I'd do differently

A short honest list, written after the bugs.

- **Sync payment saga.** Works at this scale; will break at 10× scale. Should be event-driven.
- **Hand-rolled DTOs.** I should have generated Dart and TypeScript clients from OpenAPI from day one. Three classes of bugs would have disappeared.
- **Photos in Postgres as `BYTEA`.** Easy demo, wrong for production. Belongs in object storage.
- **No feature flags.** Every change ships to everyone. Fine for diploma; not fine for real users.
- **Test coverage is uneven.** Backend is solid, mobile is sparse. Mobile needs proper ViewModel tests.



The diploma isn't the code. The diploma is the decisions I can explain.

— What I keep telling myself when I'm tempted to add more features.

What I'd build next

Things I'd add if this kept going past defense.

- **Vendor calendar sync** with Google Calendar so double-booking becomes impossible.
- **Verified-booking badge** — only customers who actually completed an event can leave reviews. Cuts review-spam to zero.
- **Embedding-based vendor recommendations** — replace today's weighted-score SQL with a real similarity model.
- **Voice input for the AI planner**, because typing on mobile while wedding-planning is brutal.
- **A proper escrow flow** with milestone release. Customers stop being scared.

Demo

I'll show you the parts that matter, in this order.

1. Switch the mobile app between Kazakh, Russian, English.
2. Ask the AI to plan an event. Watch it actually call the API.
3. Book one of the vendors it suggested. Pay with a test card.
4. Catch the push notification, deep-link into the booking detail.
5. Log in as the vendor. The booking is there, the dashboard updated.
6. Show the `docker compose logs` of the saga in action.

Eight minutes if I don't ramble. Ten if I do.

What I want you to walk away with

the AI planner isn't a wrapper

Questions?

I'd rather you ask the awkward ones now than discover them in production.

github.com/Bahaidahar/diploma · katiev2802@icloud.com